

347. Top K Frequent Elements

Medium · Array · Hash Table · Bucket Sort · Sorting · Heap

Approach — Bucket Sort by Frequency

Sorting elements by frequency normally costs $O(n \log n)$, but since frequencies are bounded by n (the array length), we can use **bucket sort** instead. Each bucket index represents a frequency count, and the bucket stores every number that occurs exactly that many times.

First, `hashMap` counts the occurrences of each number while tracking the maximum frequency `mx` seen so far. Then a `bucket` array of size `mx+2` is created, where `bucket[v]` holds every number whose frequency equals `v`. Finally, the buckets are scanned from highest frequency to lowest, collecting numbers until `k` of them have been picked.

Key Insight: Because frequency values are naturally bounded between 1 and n , bucket sort places elements directly into frequency slots in $O(n)$, avoiding the need for comparison-based sorting or a heap altogether.

Step-by-step Breakdown

Step 1 — Count frequencies and track the maximum

Iterate through `nums`, incrementing `hashMap[n]` for each number `n`. While doing so, update `mx` to the highest frequency encountered using `mx = max(++hashMap[n], mx)`.

Step 2 — Build frequency buckets

Create a 2D `bucket` array of size `mx+2` (an extra slot to safely index up to `mx`). For each (number, frequency) pair `[k, v]` in `hashMap`, push the number `k` into `bucket[v]` — grouping all numbers that share the same frequency.

Step 3 — Traverse buckets from highest to lowest frequency

Starting from the largest index of `bucket` down to 0, visit each non-empty bucket. Numbers in higher-indexed buckets have higher frequencies, so this traversal naturally yields elements in decreasing order of frequency.

Step 4 — Collect exactly `k` elements

For each number in the current bucket, append it to `ans` and decrement `k`. As soon as `k <= 0`, stop collecting — both inner and outer loops terminate once `k` elements have been gathered. Return `ans`.

Solution Code

```
1  class Solution {
2  public:
3      vector<int> topKFrequent(vector<int>& nums, int k) {
4          unordered_map<int, int> hashMap;
5          int mx = 0;
6          for(auto &n: nums) mx = max(++hashMap[n], mx);
7
8          vector<vector<int>> bucket(mx+2);
9          for(auto &[k, v]: hashMap) bucket[v].push_back(k);
10
11         vector<int> ans;
12         for(int i = bucket.size()-1; i >= 0 && k > 0; i--)
13         {
14             for(auto &n: bucket[i])
15             {
16                 k--;
17                 ans.push_back(n);
18                 if(k <= 0) break;
19             }
20         }
21         return ans;
22     }
23 };
```

Complexity Analysis

TIME COMPLEXITY	SPACE COMPLEXITY
$O(n)$	$O(n)$

Counting frequencies is $O(n)$. Building buckets is $O(n)$ since each number is placed into exactly one bucket. The final traversal visits at most n elements total. Overall: $O(n)$.

The hash map stores up to n distinct numbers, and the bucket array holds all n numbers distributed across at most $n+2$ slots — $O(n)$ total space.